

Soccer Vision - Team MegaHertz

Elettra Robotics Lab (ERL) & IIS L. Cobianchi, Verbania

ITALY

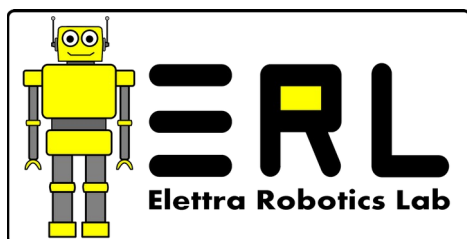


Table of Contents

1. Introduction and robot's initial overview

1.1. *I/O Pinout of **Viper X** main board*

2. Robot's general logic and behavior

2.1. *Attack and defense strategies*

2.2. *Attack maneuvers*

3. Electronic hardware

3.1. *Custom PCBs*

3.1.1. Kicker Board

3.1.2. Logic Board

3.1.3. Power Board

3.2. *Sensors and measuring units*

3.2.1. IMU

3.2.2. OpenMV Camera

3.2.3. Lidar sensors

3.2.4. Custom line sensors

4. Mechanical and structural hardware

4.1. *Custom omnidirectional wheels*

4.2. *Chassis*

4.3. *Dribbler and kicker*

4.4. *Mirror*

4.5. *Robots Design*

4.6. *Omnidirectional movement*

5. Software

5.1. *Camera*

5.2. Main MCU

5.2.1. Communications management

5.2.2. Ball following algorithm

5.3. Secondary MCU

6. Gallery

Team members

- Giraldo Luca
- Cappelli Lorenzo
- Nicò Alessandro
- Moroni Francesco

Mentors

- Raimondo Sgro'
- Vellone Daniel
- Cova Pietro

Team Introduction

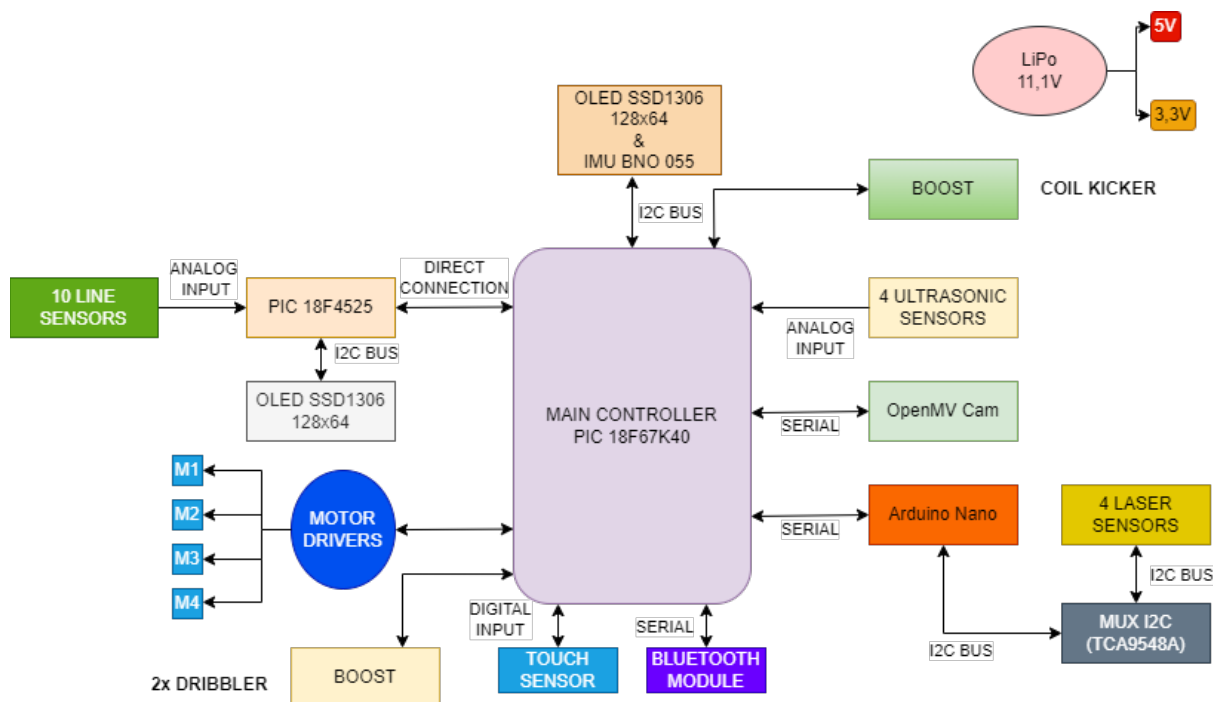
We are the Italian soccer open team **MegaHertz**. Our team consists of four students in their 4th and 5th year of high school at the **Lorenzo Cobianchi** Institute, in **Verbania**, following the *electronics systems design* course. Our school, **IIS L. Cobianchi**, hosted the **2024 edition of the Robocup Italian Competition**. Thanks to this experience the whole school got a bit closer to the whole world of robotics and its competitions. Our team has been assisted by professor Raimond Sgro' and past members Daniel Vellone and Pietro Cova whose insight and experience with the past competitions helped us get prepared for **Vienna**. Our team is organized as follows:

-
- Giraldo Luca : **Main MCU software design.** Camera vision, attack strategies
 - Nicò Alessandro: **CAD Design.** Overall robot structure, omni wheel design and production
 - Moroni Francesco: **Electronics Design.** Custom PCB design, dribbler design and production
 - Cappelli Lorenzo: **Main MCU & Secondaries MCU software design.** Line management algorithm, defense strategies, sensors management, documentation and poster.

1. Introduction and robot's initial overview

Viper IX and **Viper X** are two robots resulting from many tests and previous versions. They are very similar to each other and neither of them has a fixed role as an attacker or defender, so in case of breakdowns or penalties, they can take on each other's role. They are not entirely identical; Following extensive testing on **Viper IV**, we addressed its weak points and integrated those improvements into **Viper X**.

Both robots are equipped with a double dribbler, a highly complex hardware choice that severely limits the available space. This forced us to develop the smallest possible PCBs and meticulously detailed CAD designs. We also tested a single-dribbler robot, **Viper VII**, which has very different features compared to the ones brought to the competition. These features would be very interesting to combine with a double-roller version of Viper, but unfortunately, **Viper VII** is still in the experimental stage. In the future, we aim to explore new strategies based on the **combination** of these two robot types.



Viper X and Viper IX Block Diagram

Our robots follow a hierarchy structure where the top of it is embodied by the main MCU : **PIC18F67K40**. This 8 bit MCU runs at *64 MHz* and receives the information using multiple UART ports and also direct connections. The second most important MCU is the **OpenMV H7 Plus** camera board. This powerful board detects the ball and the goals using a microPython algorithm and sends their distance and direction to the main controller via a custom made UART protocol. Next we have the second MCU on the main logic board, the **PIC18F4525**, whose purpose is to read the analog custom made line sensors and inform the central unit when a line is crossed through a *Low Priority Interrupt* and then sends which sensor has been activated using one of the 5 direct lines. It's also important to note that the robots use a removable bluetooth module which is used in the robots strategies and also for debugging purposes using a computer.

Other peripherals that the logic board manages are :

- Four **lidar sensors**
- Two **Optical touch trigger**
- Two **Oled SSD1306 Displays**
- One **BNO055 IMU**
- One **Kicker control board**
- One **OpenMV H7 PLUS**

The main logic board is also populated with plenty of LEDs and a selectors used for debug purpose tactics switching during matches. This allows us to enable certain maneuvers based on what we think will be more impactful during the match.

1.1. I/O Pinout of Viper X main board

PIN	I/O	Function	PIN	I/O	Function
RA0	AI	liDx	RA6	AI	RULLO_SENS
RA1	AI	liSx	RA7	DO	FREE
RA2	AI	liFront			
RA3	AI	liRear			
RA4	DI	SELAUX1	RD0	DO	M1_A
RA5	AI	VEL_POT	RD1	DO	M1_B
RB0	DI	INTERRUPT_LINEA			
RB1	DI	TX4	RD2	DO	M2_A
RB2	DI	P_START	RD3	DO	M2_B
RB3	DI	AD_SEL	RD4	DO	M3_A
RB4	DI	ARIETE	RD5	DO	M3_B
RB5	DI	RX4	RD6	DO	M4_A
RB6	DI	LIN_AV_SX	RD6	DO	M4_A
RB7	DI	LIN_IND	RD7	DO	M4_B
RC0	DO	PWM_RULLO			
RC1	DO	PWM_RULLO_2	RE0	DO	D1 LED_ON
RC2	DO	PWM_MOT_2	RE1	DO	D2 LED_PWR
RC3	DO	SCL	RE2	DO	D3 LED_UL

RC4	DO	SDA	RE3	DO	RULLO_REAR
RC5	DO	BUZZ	RE4	DI	SW2 P_BUS
RC6	DO	PWM_MOT_3	RE5	DO	PWM_MOT_4
RC7	DO	PWM_MOT_1	RE6	DI	RX3
RF0	DI	LIN_AV_DX	RE7	DO	TX3
RF1	DI	MUX_RESET	RG0	DI	RX2
RF2	DI	TOCCO_FRONT	RG1	DI	LIN_DX
RF3	DI	TOCCO_REAR	RG2	DI	LIN_SX
RF4	DI	LIN_PRIO_1	RG3	DO	TX2
RF5	DO	RUN	RG4	DI	FREE
RF6	DI	P_AUX	RG6	DO	TX5
RF7	DO	KICK	RG7	DO	RESET_OLED
RH0	DI	SELAUX1			
RH1	DO	LED_AUX			
RH3	DI	FREE			

2. Robot's general logic and behavior

As we said before the two robots have different roles on the field. While developing their behavior we pictured in our minds the three main different scenarios that each robot could encounter, then we implemented each robot's "personality" according to their role and to the current scenario they'd be put in.

2.1 Attack and defense strategies

Since each robot is able to communicate to the other using their **bluetooth module** we planned to make them share important information about

their state. Since we defined “IX” as the defender and “X” as the striker it would be useful, firstly, to communicate the “I’m Alive” character to the other robot. In this way in the event one robot exits out of bounds, thus has to be removed from the field for one minute, the other one stops receiving the character. This triggers a condition which commands the robot to roll back into defense in case it doesn’t have sight of the ball despite their role. Just this simple control helps us receive less goals since we have almost always a robot actively defending from opponents scores.

Next the robots share their “**Ball status**” which essentially tells the other one if it has the ball in sight or in the dribbler. With this information the other robot falls back in defense while the one with the ball attempts to score.

Lastly when both robots don’t see the ball we have defined their behavior based on the **A/D** (Attack / Defense) **selector**. The robot that is marked as *defender* will place itself in front of the ally goal. Meanwhile the robot who is defined as an *attacker* will drive towards the center of the field, this should give it a higher chance of spotting the ball, thus start to chase it. These maneuvers are performed with the use of distance measures such as Lidar sensors. We also use the camera goals information to center our robot in front of the ally goal.

The **A/D** selector also influences the role communicated to the other robot.

2.2 Attack maneuvers

Some maneuvers that we have implemented for scoring are the following :

- **Standard/Torsion shot :**

The robot moves towards the opponent's goal after it has caught the ball with the dribbler. If the alignment is right it shoots right away, otherwise it twists itself towards the center of the opponents goal



while retaining the ball with the dribbler. Once it is aligned correctly the kicker gets activated.

- **Catch and fall back :**

If the ball is in one of the opponents corner the robot will try to catch it with its dribbler. If he succeeds it will back up to a distance where it can safely see the opponent's goal. If its vision is disturbed by the presence of an opponent the robot will zoom to the opposite side of the goal and then attempt a shot. Here is an extract of the code for this maneuver.



```
if(direzionePortaAttacco >= 60 && distanzaPortaAttacco <= 70 && TOCCO == 1 ){
    vel = velBase - 0.5;
    centra_verso_assoluto(0);
    dir_180();
    __delay_ms(400);
    fermo();
    centra_verso_assoluto(40); // da vedere con rullo nuovo
    leggi_sensori_distanza();
    if(ulFront >= 30 && TOCCO == 1){
        rullo(0);
        __delay_ms(50);
        sparo();
    }else {
        dir_45();
        if(direzionePortaAttacco >=350 && direzionePortaAttacco <= 10 && TOCCO ==
        // __delay_ms(1000);
        rullo(0);
        __delay_ms(50);
        slancio_sparo();
        // goto CICLO;

    }
}
goto CICLO;
}

if(direzionePortaAttacco <= 300 && distanzaPortaAttacco <= 70 && TOCCO == 1 ) {
    vel = velBase - 0.5;
    centra_verso_assoluto(0);
    dir_180();
    __delay_ms(400);
    fermo();
    centra_verso_assoluto(310); // da vedere con rullo nuovo
    leggi_sensori_distanza();
    if(ulFront >= 30 && TOCCO == 1){
        rullo(0);
        __delay_ms(50);
        sparo();
    }else {
        dir_315();
        if(direzionePortaAttacco >=350 && direzionePortaAttacco <= 10 && TOCCO == 1){
            // __delay_ms(1000);
            rullo(0);
            __delay_ms(50);
            slancio_sparo();
            // goto CICLO;

        }
    }
    goto CICLO;
}
```

“Catch and fall back” maneuver’s code extract

3. Electronic hardware

The electronic hardware on our robots allow us to perform all kinds of tasks. Without these external reference points it would be nearly impossible for our robots to move in straight lines, follow the ball, decide when to shoot and when to stop before it gets out of bounds. On top of that our custom PCBs house all the microcontrollers and other hardware used by the robot to perform his actions and by us to monitor the state of each algorithm.

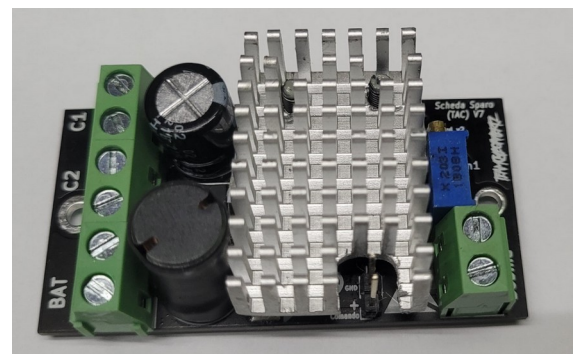
3.1. Custom PCBs

Our custom PCBs were all designed using **Kicad 8**, an open source and free EDA software perfect for beginners and more skilled designers. Through its schematic and PCB editor we were able to draw our boards from the ground up without any issues. We found plenty of plugins that aided our development process. Once the boards were ready for production we reached out to a local company named **Millennium Dataware SRL** who offered to produce them and sponsor us. Although we had a stable supplier for our PCB we still had to rely on some 3rd party services and their fast shipping for a batch of boards that wasn’t planned. The first version of the kicker board would generate a lot of electrical noise and its capacitor charging speed was low, due to a design flaw in the placement of the components.

3.1.1 Kicker Board

The **kicker boost board**, also referred to as “**TAC Board**” by us, is designed to actuate a “homemade” electromagnetic solenoid. The “**TAC board**” purpose can be split into two main actions :

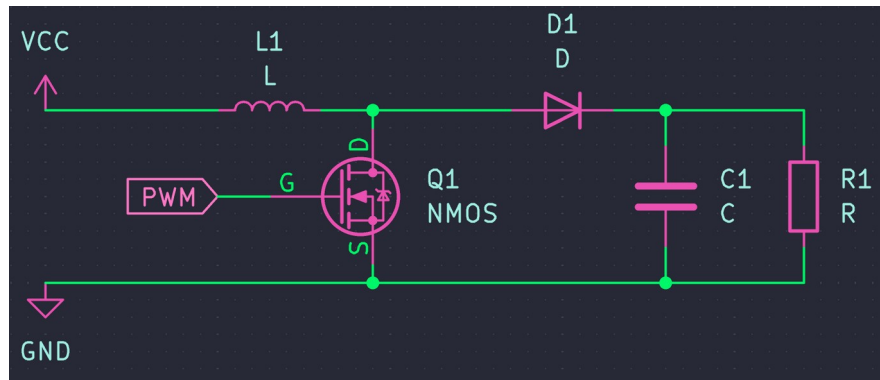
- The board design is based on the UC3843 switching regulator.



TAC V2 board's PCB and finished product

From the schematic diagram it is possible to spot the DC-DC buck converter circuit.

clarity
here is a
bone"
of the



For
sake
"bare-
version
board.

Kicker module demonstrating circuit

The circuit has two different states :

- **NMOS "Closed"**: the supply is basically shorted to ground, therefore the current increases linearly through the *inductance* L . This phenomenon causes the *inductor* to build up a *magnetic field* which also grows linearly. The duration of this state last **T1**
- **NMOS "Open"**: The current that has built up in the inductor charges the *capacitor* C with a constant current. This allows the *capacitor* to charge at a higher voltage compared to the power supply one. The diode D controls the current flow to prevent a back current to flow in the supply. The duration of this state last **T2**
- **Equation of Step-Up circuit:** $V_o = V_i \cdot \left(1 + \frac{T_2}{T_1}\right)$

Regulating **T1** and **T2** it is possible to regulate the output voltage. The input signal of the circuit is given by a **PWM**

generated signal, with variable **duty cycle**. The switching regulator is the UC3843, in the smd package.

Using the following equations we were able to calculate the output velocity of the ball should end up with, ideally.

$E = \text{energy} / C = \text{Capacitance} / V = \text{Voltage}$

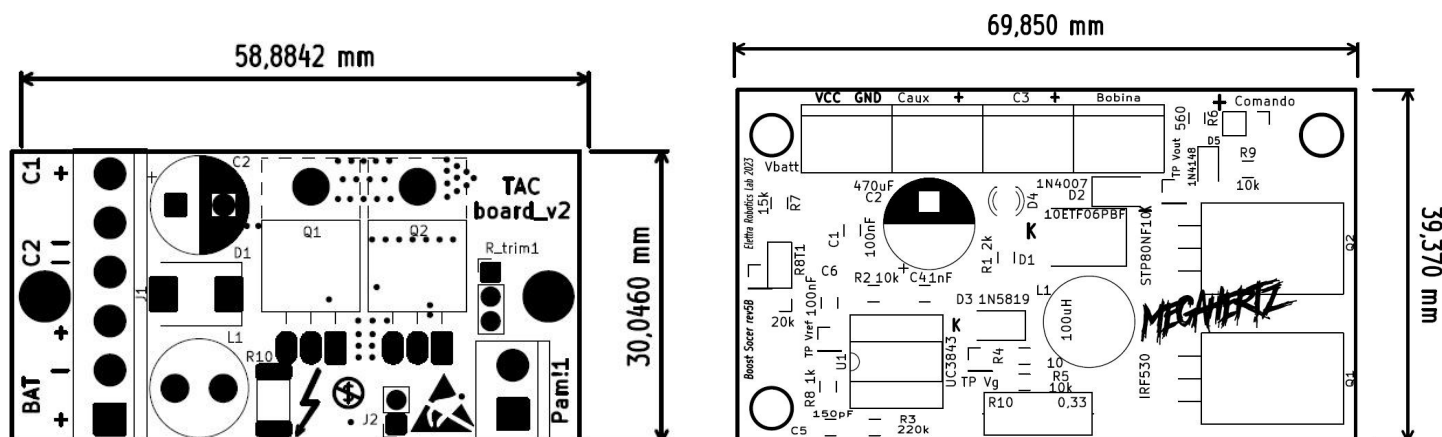
$$E = \frac{1}{2} \cdot C \cdot V^2$$

Ideally the conversion of energy from potential energy, in the capacitor, into kinetic energy in the ball should be perfect, thus the velocity of the ball should be defined as:

$$V = \sqrt{\frac{2E}{C}}$$

Since the rules limit the kicker voltage at 48 V we had to increase the capacitance of the overall system, that's why we've increased the capacitors to 9400 μF .

Lastly we wanted to share the more compact design of the **TAC Board V2**, compared to the previous kicker module. This could be achieved by pursuing a more efficient component placement.

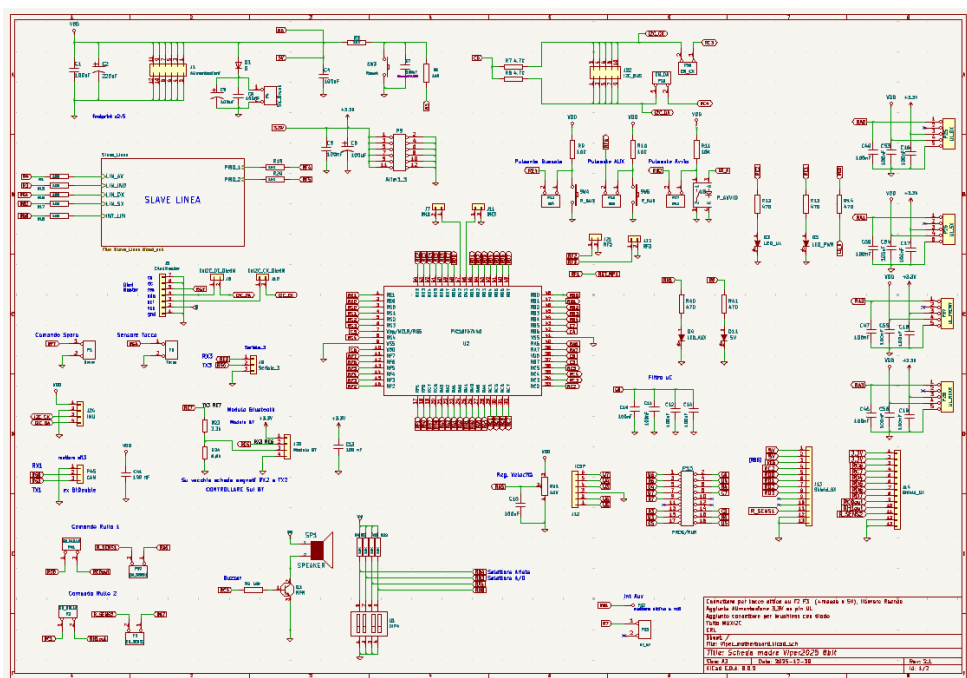


Kicker module V7(TAC V2) and V5b compared

3.1.2. Logic Board

The logic board is the heart of our robots. It houses most of the vital components of the project. Here we can find both the two microchip © 's MCUs. The primary controller runs at a frequency of 64 MHz while the secondary one runs at 4 MHz. The board also hosts serial ports for the OpenMV H7 Plus and Bluetooth boards. Last but not least there are two I2C channels. One hosted by the secondary MCU whose purpose is to display, on an OLED, the state of each line sensor, as well as calibration uses. While the other one has as its master the main microcontroller. This channel is populated by another OLED Display and by the BNO055 IMU. Lastly each MCU has an ICSP connector and switch, used for programming. This board has to cram all of this and even more in a relatively little surface. Because of this the power section of the robot has been moved to a second board because it would be impossible to fit the motor drivers in the first one. Also it helps reduce the noise that would have been certainly a problem. Moreover, the board's dimensions were reduced compared to the previous

iteration, in order to avoid obstructing the field of view of the OpenMV H7 Plus camera mounted on the robot. If we sum the number of pins available on the board, coming from both MCUs, the total amounts to almost 100 GPIO, who are all populated. This allows us to have a total control of all the hardware onboard. The board is equipped with some additional circuitry such as the ultrasonic sensors attachments, debug and strategies selectors, a buzzer and line sensors state LEDs (that also have the function of communicating the state of each line sensor from the secondary MCU to the main one through a direct connection).



Main MCU Schematic





3.1.3. Power Board

As we said in the last paragraph the final design ended up with having the power components on a different board from the logic one. This decision has its ups and downs. It is beneficial for the overall system to have this division since the electrical noise won't spread as easily in the logic circuitry. Also having a fairly “empty” board for the power allows us to have a big ground layer on the PCB which helps with heat dissipation. A big negative is that the board occupies space, which means that whole robot must get taller. This is caused by the beefy heat dissipators used for the motor drivers.

The board consists of three main structures :

1. **H Bridges command circuitry**
2. **Dribblers command circuitry**
3. **Voltage regulator circuitry**

Firstly the main purpose of this board is to control each of the four motors used for omnidirectional movement. The board presents three **ST L6207N H bridges**. Each IC can control the direction and speed of two motors. They require some external circuitry to work properly and to protect both the IC and the MCU in case of short circuit or overheat. The signals are shared between the two boards using two lines of headers that interlock when the robot is mounted.

Secondly this board controls one H bridge attached to the dribbler motor. Each dribbler is equipped with a PWM command signal and two commands for direction.

Lastly the board hosts both the 5 V and 3.3 V regulators. For both the regulators we opted for switching ones. For example we calculated the power loss on the old 5 V regulator (**L7805**) when supplied with 12 V. Knowing that our logic circuitry draws around 200 mA it's easy to find the power dissipated by the IC:

$$PD = (V_{CC} - V_{regulator}) \cdot I_{logic} = (12 - 5) \cdot 200 \cdot 10^{-3} = 1.4 \text{ W}$$

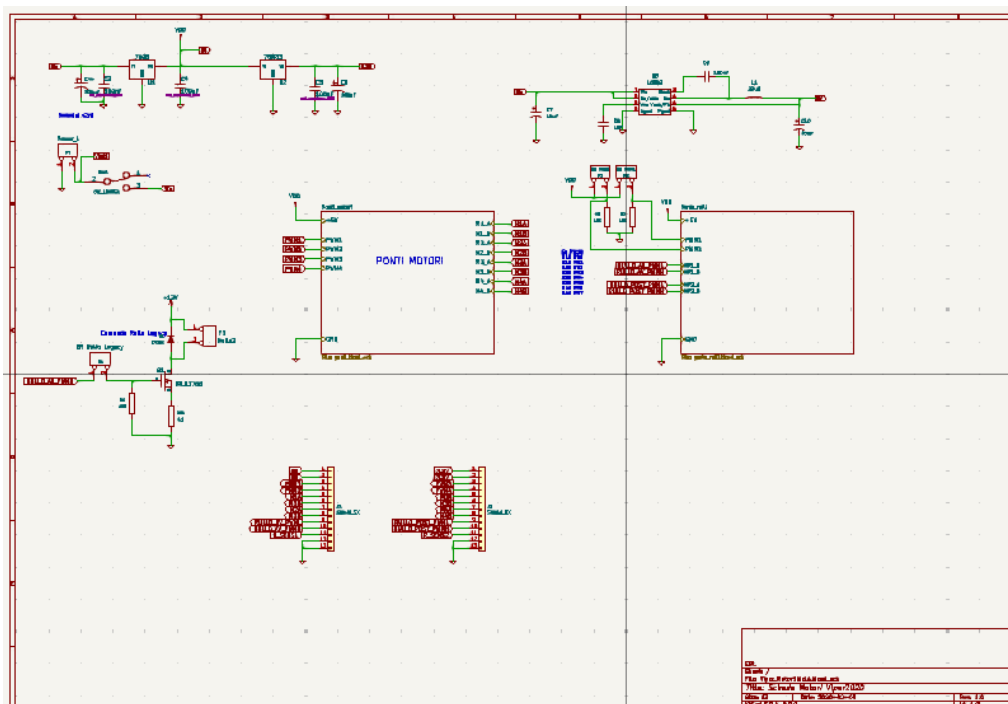
Which is a lot. Not to mention that there is a power loss of almost 60% and using this IC means that it would be mandatory to provide it with a beefy heatsink since from the datasheet it specifies that the temperature of the component rises 60° at every watt the component dissipates.

In contrast, switching step-down circuits have a higher power efficiency (even the cheapest has a power efficiency of around 70%). Also they are cheaper than **Traco Power** IC. The overall working principle of a switching IC is the same used in the kicker step-up circuit. The notable difference is the position of the MOSfet and its type. In a step-down circuit the MOSfet is a PNP one, on a high-side configuration. Like the step-up equation the step-down one is very similar with one huge difference.

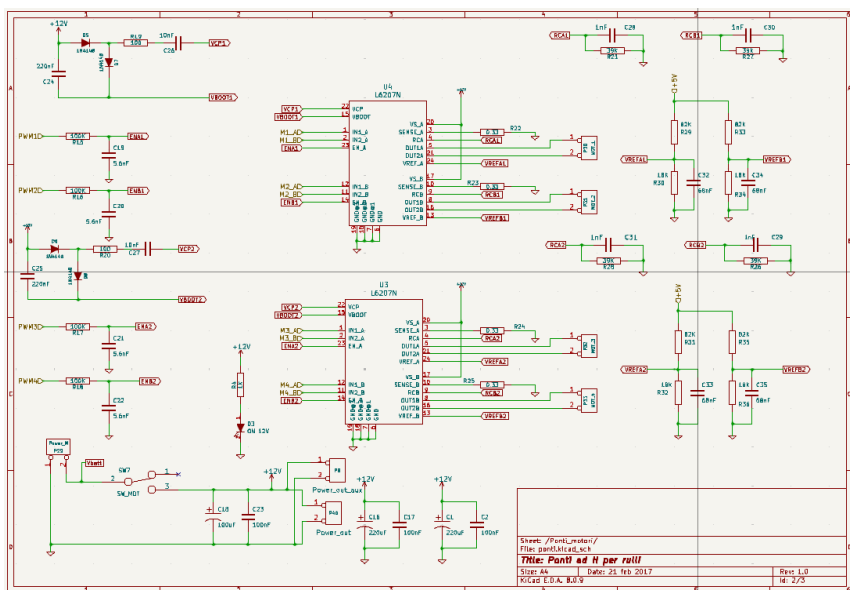
$$V_o = V_i \cdot \frac{T_1}{T_1 + T_2}$$

Where T_1 is the period of time where the supply charges the inductor, and T_2 is where the inductor is connected to ground, therefore its current decreases.

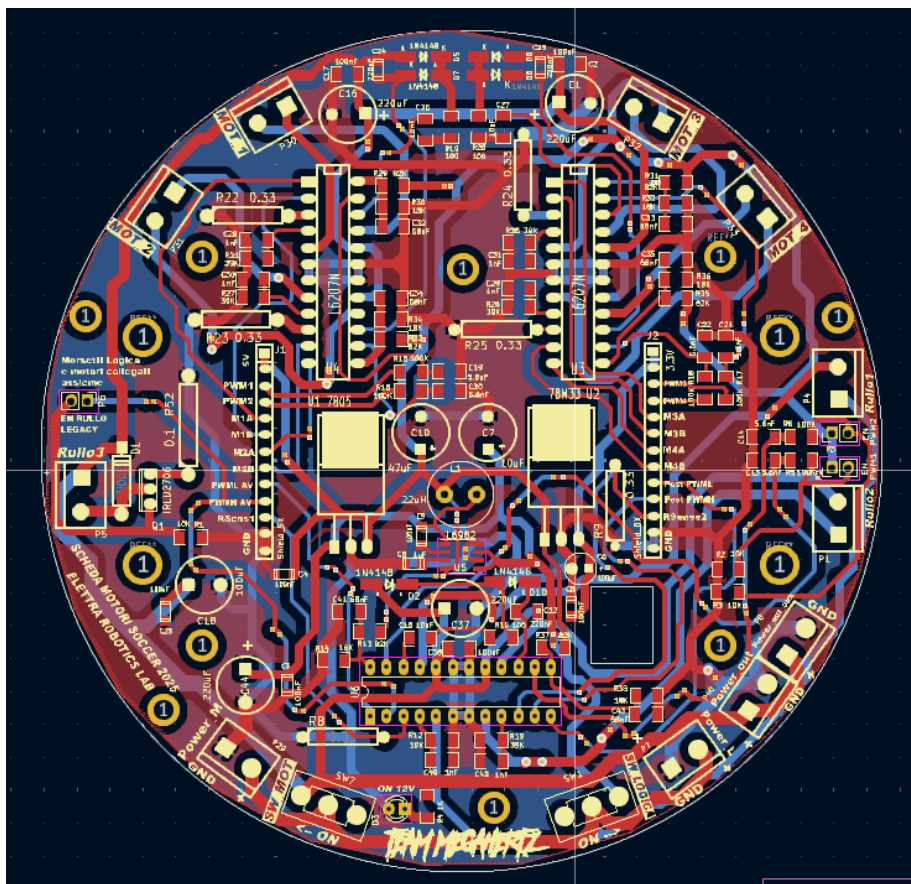
Compared to the step-up formula we can see that the term $T_1/(T_1 + T_2)$ is always less than one, so that V_o is less than V_i .



Power Board schematic, page 1



Power Board schematic, page 2



Power Board PCB

3.2. Sensors and measuring units

Viper X and **Viper IX** have plenty of sensors scattered all over their surface. They help us develop and perform actions that require precise movements. The most important measuring units are the following.

3.2.1 IMU

The Inertial Measurement Unit is a key component for the implementation of movement in general. Thanks to this device we can measure the robot's movement in 9 degrees of freedom. Inside the IMU there are 3 smaller sensors: a gyroscope, an accelerometer and lastly a magnetometer. Each of these sub components measure the change of position in 3 degrees of freedom. On a rover like our robots some of these measurements are not

important for the overall result. Also the IMU is a big upgrade compared to the magnetometer used on our robots many iterations prior, since this sensor is faster and less subject to noise caused by external devices.

On our robots we use the **BNO055** which gives us the angle measured from the initial “north” position to the actual orientation of the robot. The initial north position is set when the cycle starts, pointing at the opponent goal. The feedback given by this peripheral is “fed” into an algorithm which uses a PID controller whose purpose is to compensate for any misalignment

```
void leggi_bussola(void) {  
    // Funzione per la lettura della bussola IMU BNO055  
    // La bussola deve prima essere inizializzata -> init_BNO055()  
  
    writeI2C(BNO055_ADDR_WRITE, 0x1A);  
    app_l = readI2C(BNO055_ADDR_READ); // (char)  
  
    writeI2C(BNO055_ADDR_WRITE, 0x1B);  
    app_h = readI2C(BNO055_ADDR_READ); // (char)  
    app_h = app_h << 8;  
  
    bus = ((app_l + (app_h)) >> 4); // 0-359  
    //bus255 = ((app_l + (app_h)) / 22); // 0-255  
} //end leggi_bussola()
```

errors. The PID action is passed to the PWMs equations which control the motion of the system.

IMU Reading Function

3.2.2 OpenMV Camera

For the camera we decided to use an **OpenMV H7 plus**. This powerful module ensures us to have a stable FPS rate and also a fast enough processor that takes care of all the image recognition software. We pointed the camera towards the roof of the robots. There a mirror is mounted with its vertex point aligned to the camera objective. The reflection on the mirror is captured by the camera, obtaining a 360° view. The camera communicates to the logic board using a serial port.



3.2.3 Lidar sensors

The Analog lidar sensors are the primary distance measurement units available on our robots. The robot mounts four of these sensors, one on each side, providing full 360° coverage around its perimeter. Each sensor guarantees a reliable range of up to *1.5 meters*, which is more than sufficient for all the maneuvers performed on the field. Their fast reading rate and adequate precision make them ideal for quick distance checks: whenever the robot needs to measure its distance from a wall, an opponent, or a reference point, the lidar sensors are the go-to solution. They are used extensively across all movement and positioning routines. Since they are analog sensors we have to filter the noise that could corrupt the precious data they provide. That's why we came up with two solutions to the noise problem: one of them is handled by software which we'll talk about in the next paragraphs, while the other is handled through hardware.

3.2.4 Custom line sensors

Both **Viper X** and **Viper IX** are equipped with 10 line sensors. The sensors are positioned, in pairs, in a pentagon shape having the right, rear and left sensors form a cross, while the last two pairs are positioned at an angle of 15 and -15 compared to the robot forward center. Each vertex of the pentagon has an internal and external sensor. With this configuration we can firstly identify the presence of the line using the external sensors, then once the robot gets too out of bounds and triggers an internal sensor a reentry maneuver will be performed.

Each line sensor consists of a small PCB with 2 pairs formed by an LED and a phototransistor one next to the other. The light difference between the field and the line will cause the phototransistor to change his voltage value. This analog value is read for each pair by the secondary **MCU** using an analog pin. All the line sensors are mounted on custom made 3D printed support bolted to the bottom frame of the robot. To further improve the precision of the line interrupt, we are introducing a new line sensor on the

front of the robots. In future iterations we also plan to add a single sensor mounted horizontally on the rear, in order to extend the same improvement to the back of the robot as well.



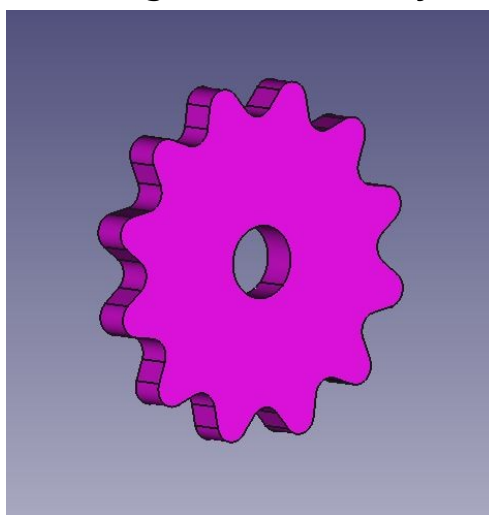
Line sensor board near the 3D printed support

4. Mechanical structure and hardware

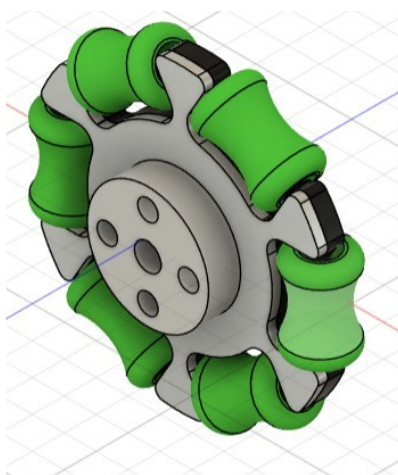
The main goal for mechanical improvements this year was to decrease the height of our robots compared to the previous ones, in order to enhance vision. To make this possible we had to reduce the distance between the two main frames, which is no easy task due to the presence of the dribbler's motor and structure. Since we wanted to have two dribblers on **Viper X** and **Viper IX** a smaller kicker module was designed and mounted vertically to house the second dribbler. Another task for this year mechanics was to enlarge the distance between the wheels for better stability, and also produce a new kind of omni wheel with enhanced grip.

4.1. Custom omnidirectional wheels

We have updated the robot's design by replacing the 24-cogwheel assembly with new wheels featuring silicone rubber inserts, which, combined with our specific motor configuration, significantly enhances both grip and overall responsiveness. Additionally, the external protective housings are now manufactured from in-house CNC-milled aluminum to ensure superior structural strength and durability.



tooth-shaped washer

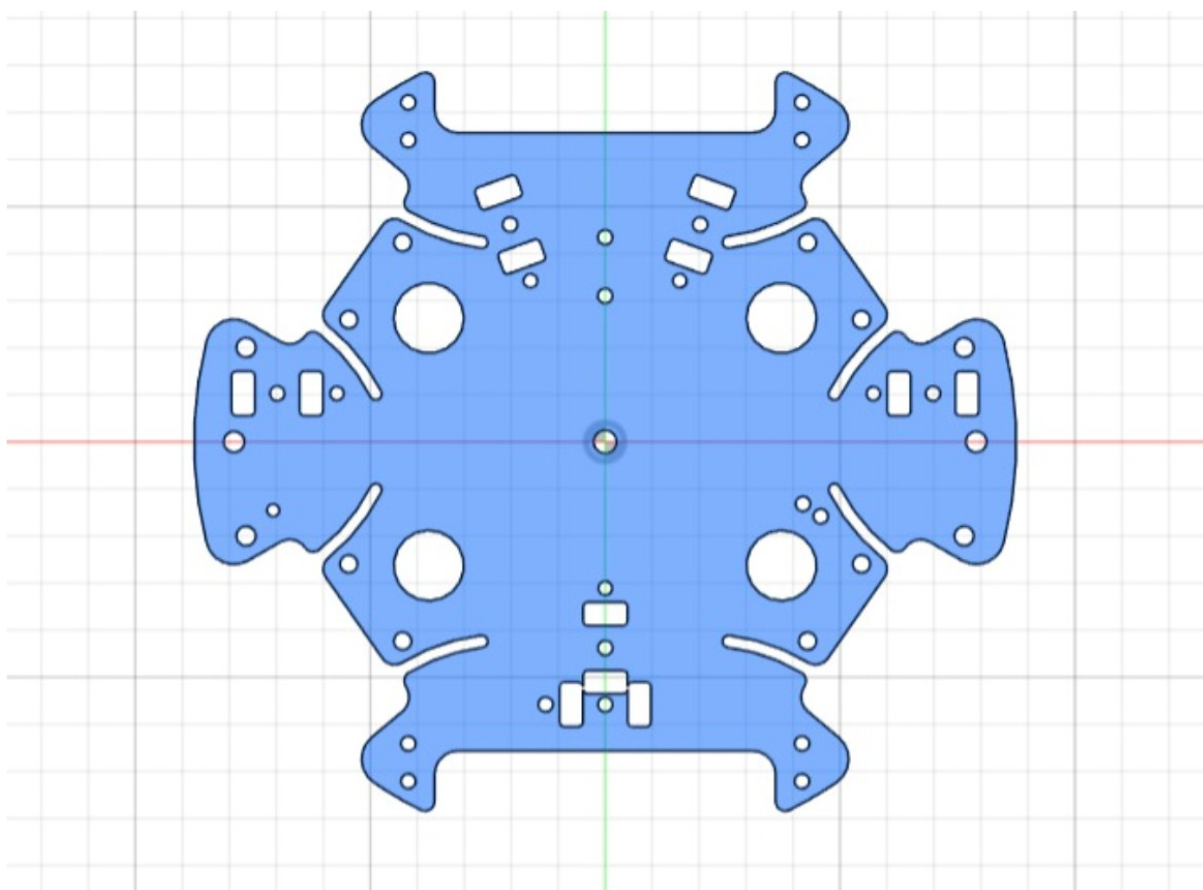


Rubber wheel

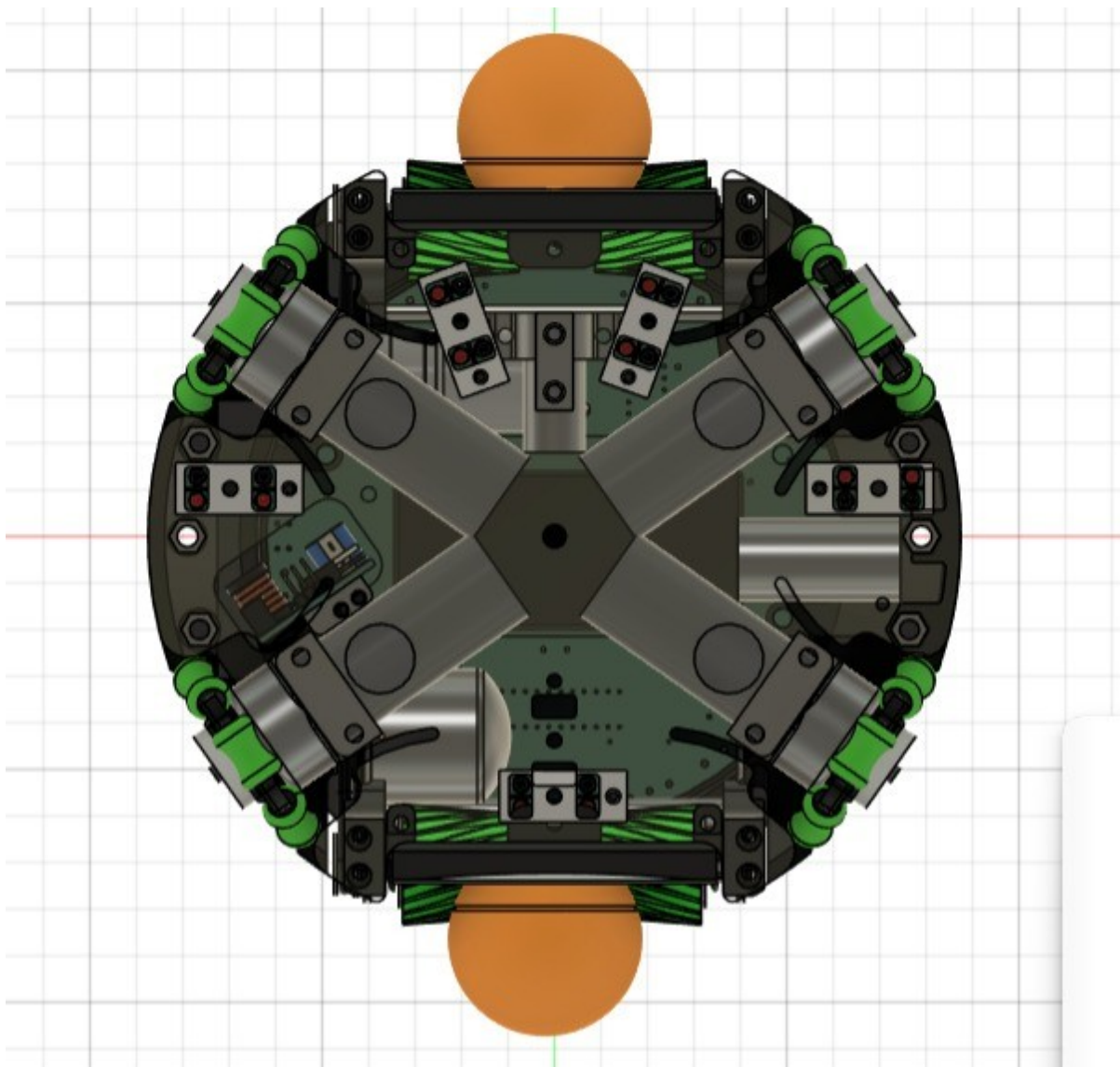
4.2. Chassis

In the design of the frame we kept 4 motors with a 55° angle, to give more power to the robot. The kicker coil is fixed this time on the upper frame to interfere less with the motors. The line sensors maintain a pentagonal arrangement with two phototransistors for each block. This should allow us to obtain the crossing direction of the white line.

to improve the robot's grip we designed curved suspensions, thus exploiting the elasticity of the lower polycarbonate frame. In order to protect the wheels, we also designed bumpers, as we can see in the image below.



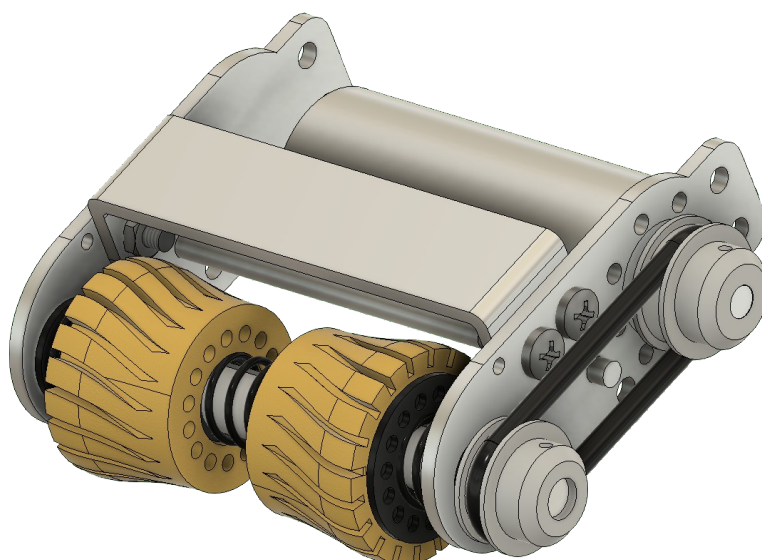
The lower frame design, with curved suspensions



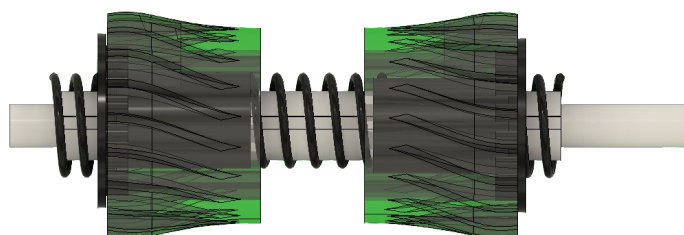
Bottom view of the robot, you can see the bumpers and the arrangement of the line sensors.

4.3. Dribbler and kicker

Dribbler and kicker are crucial to football strategy, so we put a lot of thought into the design. The Dribbler is driven by a DC geared motor at 2700 RPM, the transmission is belt driven, it has two degrees of freedom to better adapt to the ball. We made the rubbers with two-component silicone rubber, formed using 3D printed molds.

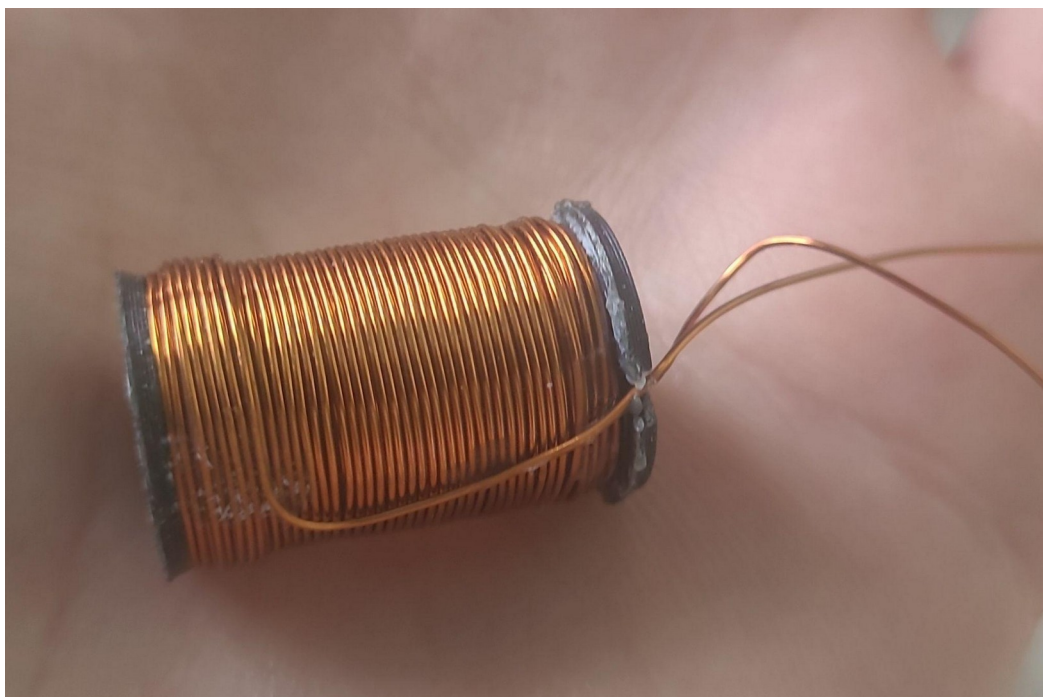


Dribbler with drive belt. it can move on a pivot that raises it on the z axis



The rubbers can move horizontally, this gives the second degree of freedom to the dribbler.

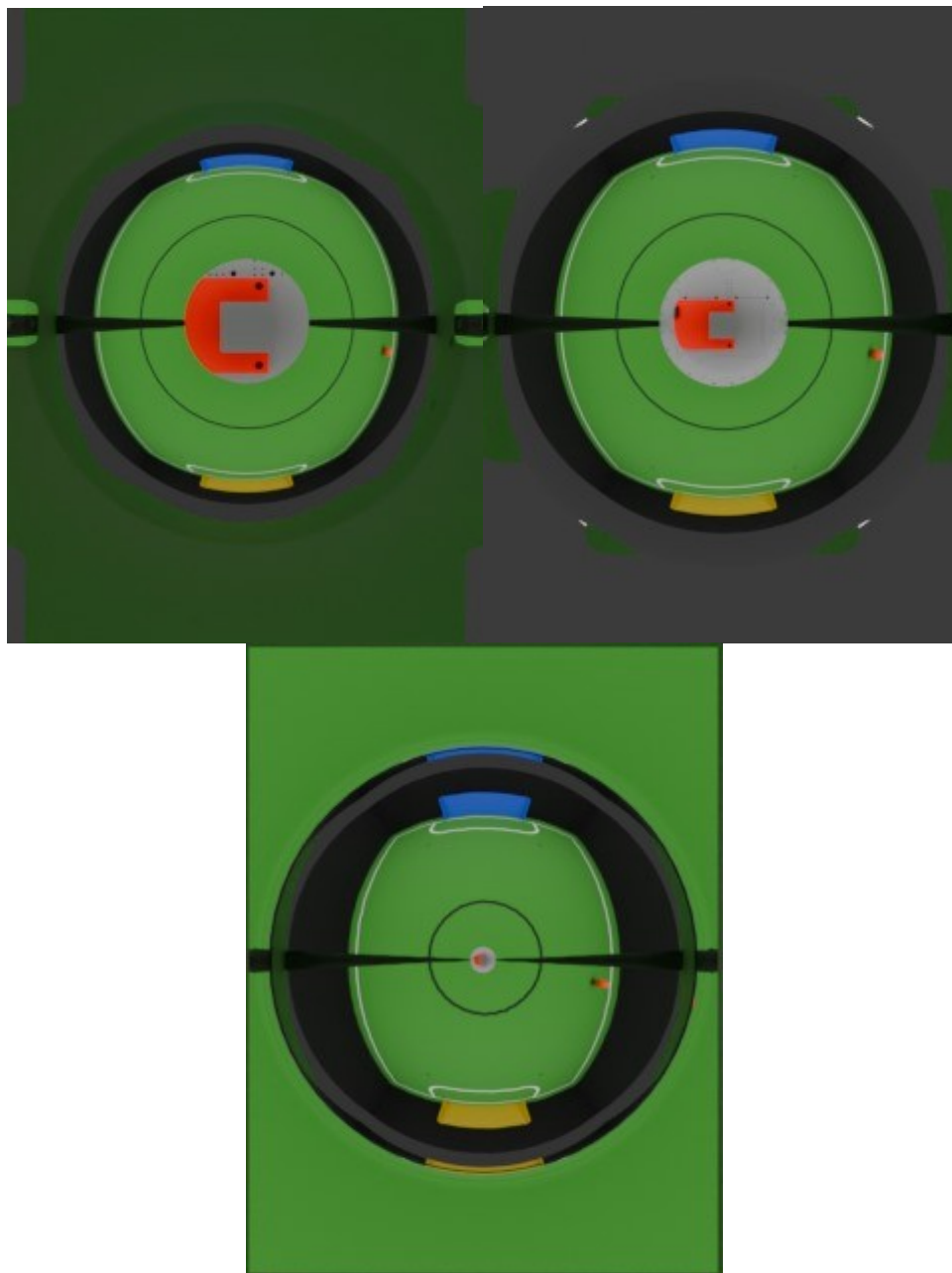
The Kicker was made with a homemade coil, the piston is made half in iron and half in no magnetic material, the iron part is positioned external and kept in pose by a spring. When current flows through the coil, the iron portion is pushed inside the coil, so that the piston can kick the ball. The force of the piston is due basically by two factors: current and magnetic field. In order to increase the power of the kicker we decided to reduce the resistance of the coil, from 1.3Ω / 1.5Ω to 0.7Ω . So when the MosFet of the kicker module is driven On by the **MCU**, the burst current is: $\frac{48V}{0.7\Omega} = 68A$. This means that we have to use an appropriate MosFet. Compared to last year, the solenoid coil has been repositioned further back on the frame in order to increase the shooting range.



Our homemade coil. We used a 0.4mm diameter copper wire

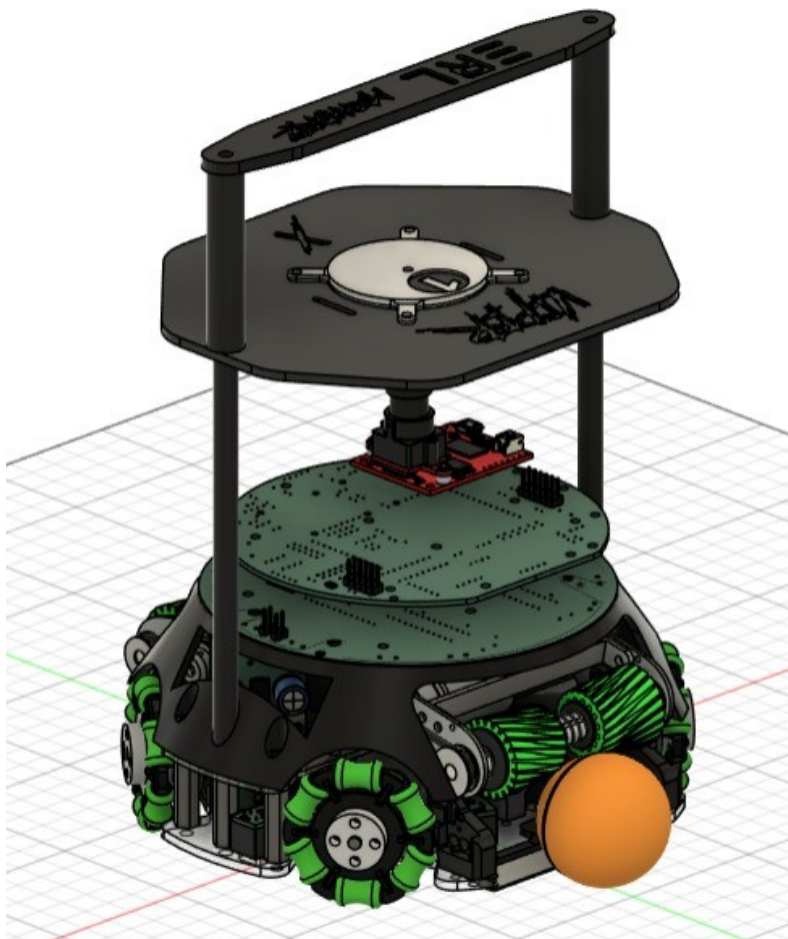
4.4. Mirror

Vision is the main feature that the robot must have, so we spent a lot of time designing the mirror, trying different profiles. To simulate the behavior, we created a virtual field in the software blender, below are the results of the simulation on the different mirror profiles.



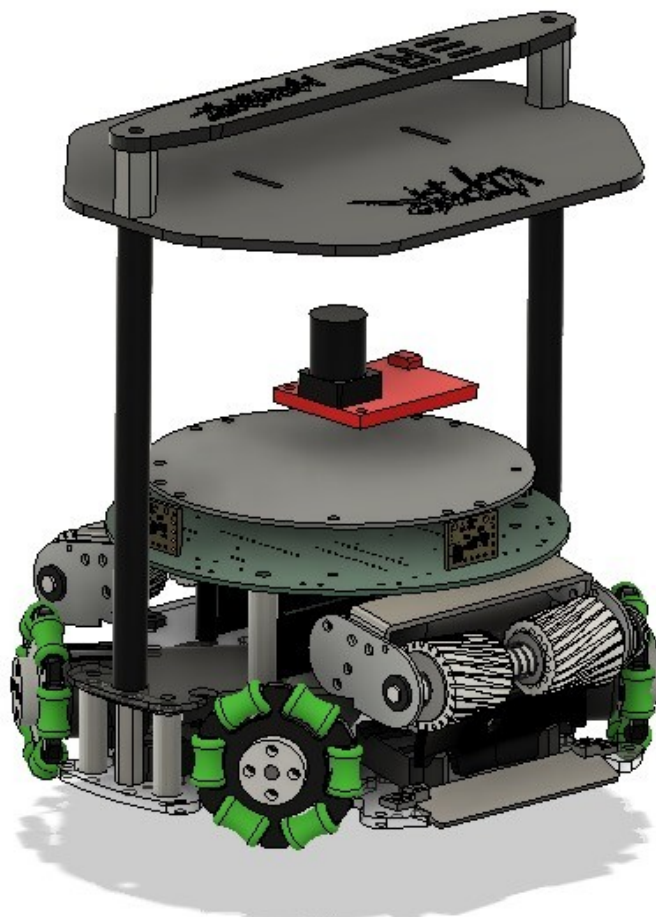
Different simulations obtained in Blender with different mirror profiles





Design of Viper X

4.5. Robots design

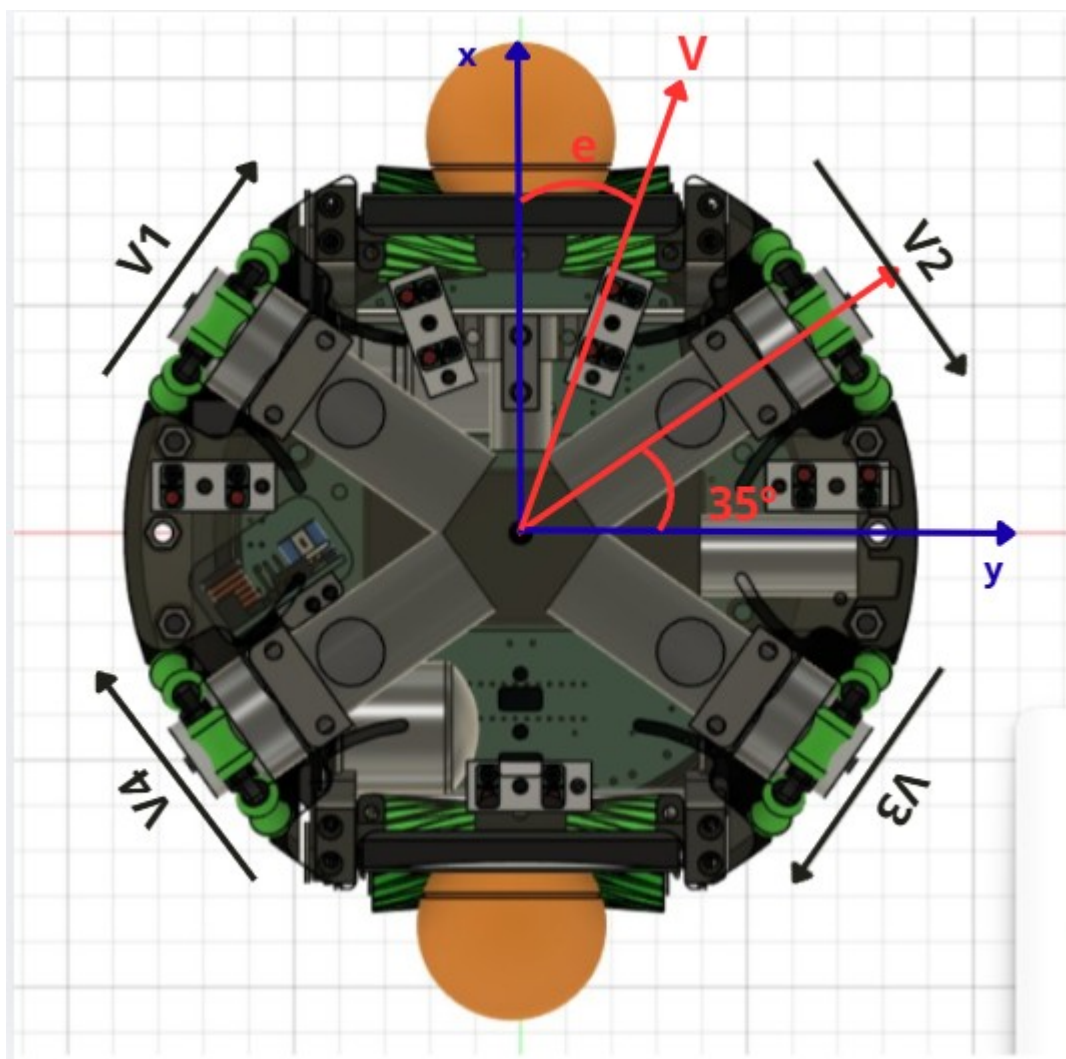


Design of Viper IX

4.6. Omnidirectional movement

The Omnidirectional movement is obtained by composing the four speed vectors of the motors. If we assume that the angle formed by of each motor with x and y axis is 55 degrees, we can say that:

$$\sin(55^\circ)=0.819, \cos(55^\circ)=0.574$$



By decomposing the four vectors along the two axes, we obtain the following equations:

$$V_x = (V_1 - V_2 - V_3 + V_4) \cos(35^\circ)$$

$$V_y = (V_1 + V_2 - V_3 - V_4) \sin(55^\circ)$$

These equations are not directly invertible; therefore, an additional constraint is required. We can impose that the robot does not rotate around its own axis by requiring the sum of the motor velocities to be zero:

$$0 = V_1 + V_2 + V_3 + V_4$$

Since the system still contains four unknowns, the equations cannot be solved directly. To obtain the inverse kinematics, matrix calculations must be used.

If we assume that the resulting velocity vector V , composed of V_x and V_y , forms an angle θ with respect to the x-axis, its components can be written as:

$$V_x = V \cos \theta$$

$$V_y = V \sin \theta$$

Using matrix calculations, we can derive the equations for each motor:

$$V_1 = V_x \sin(55) + V_y \sin(35)$$

$$V_2 = V_x \sin(55) + V_y \sin(35)$$

$$V_3 = V_x \sin(55) - V_y \sin(35)$$

$$V_4 = V_x \sin(55) - V_y \sin(35)$$

Finally, we can assume that the angular velocity of each motor is directly proportional to the PWM signal applied to it.

5. Software

Each **Viper** robot has multiple codes that had to be managed by our team. Since our robots use four different processing units, each written in a different development software and with different programming languages, we want to split the explanation by processing units, rather than by strategies or competence areas.

5.1. Camera

The **H7 plus** camera is programmed in micropython. It uses some libraries and functions especially made for image processing. We first worked on ball detection. To find the ball we scanned each frame in search of some “blobs” with the right color. The color’s thresholds are generated manually using the OpenMV calibration tool. Once the camera finds some groups of pixels with the right color it starts excluding the blobs that aren’t big enough or are too big. These two thresholds are hard coded by us after many trials and errors, until we found some safe values. Once the ball detection started working we pushed on with the ball position and distance functions. To find the ball angle in reference to the camera center we used a more complex equation:

$$\left(\arctan\left(\frac{Ball_Y}{Ball_X}\right) \cdot \frac{180}{\pi} \right) - 180$$

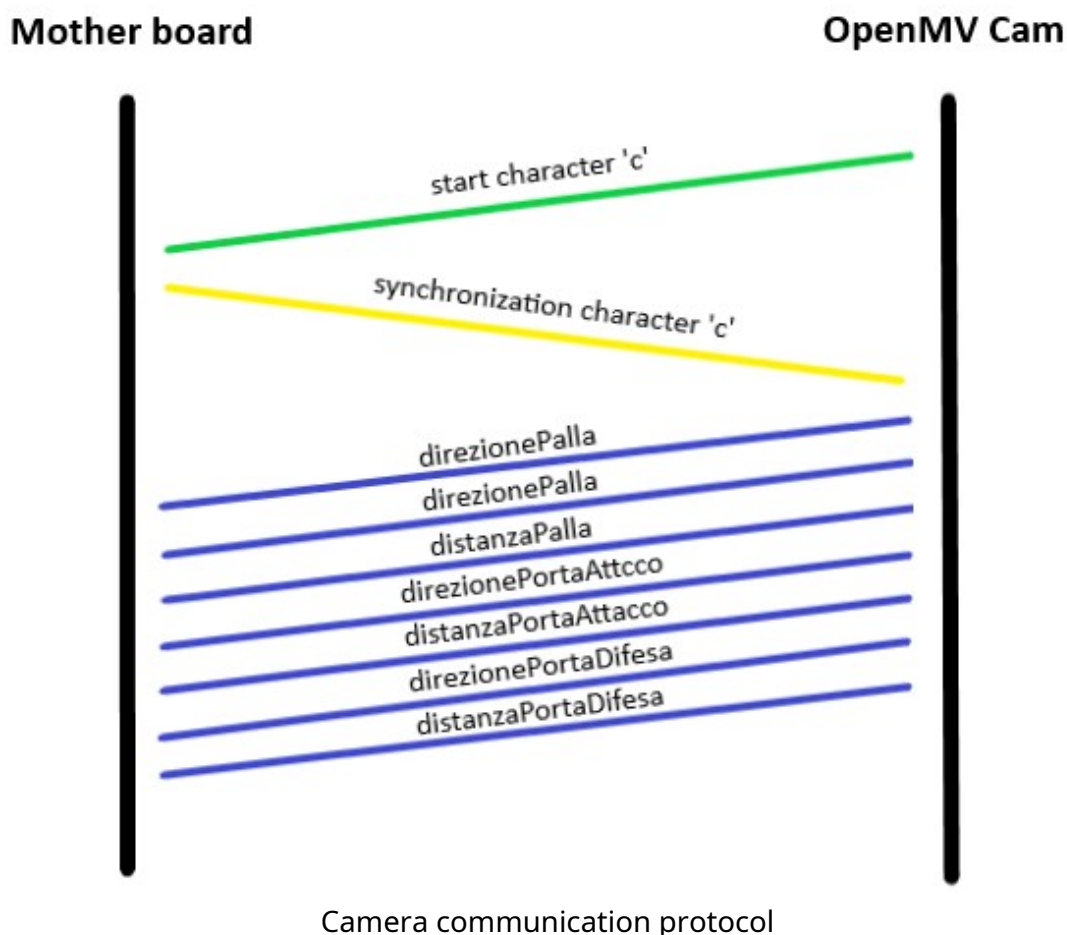
The ball's X and Y coordinates are calculated by finding the blob center and then addressing its position in the pixel grid.

In contrast to calculate the distance we measure the distance between the center of the camera and the blob position. Since the mirror is a curve the distance doesn't increase linearly but more in a logarithmic way. We lock the distance at 0 when the ball is spotted in the small mirrors used for close up view.

For the goals we use the same algorithm to calculate their presence and direction. For the distance calculation we use the area of the blob. With this method we can accurately calculate the distance of each goal until the middle of the field, past that the camera loses track of the most distant goal.

Each object tracked by the camera has a pair of values : their distance and their direction. To communicate these values to the main controller we use a custom UART protocol. Since the main controller is an 8 bit one each packet communicated must be of that size. This is a problem if we want to communicate the direction of the objects on a $0 \rightarrow 360$ degree base. That is why we opted for this solution : the goal's direction will be transmitted on a 8 bit base, so we firstly "compress" dividing the value calculated by the OpenMV in such a way that the maximum value is 255. Then the value gets transmitted and "extracted" from the main MCU by multiplying the received data for the same value as before. Doing this step implies that we lose some precision but it's acceptable since we don't need that extra precision anyway. A different approach has been taken for the ball's direction. Since this value is more important we want to send it whole. That is why we decided to send the data in two 8 bits packets. Although most of the 16 bits here are unutilized we need the extra precision to have a more fluid ball chasing movement. The camera protocol works on the main microcontroller high priority interrupt, thus it immediately transmits the values whenever they are ready. The protocol starts with a

“handshake” character. Once the handshake has occurred the camera starts the transmission.



5.2. Main MCU

5.2.1. Communications management

The main MCU has to manage the data that it receives from different devices and with different protocols. The higher priority is given to the camera's data which communication interrupts the process with the highest priority. In this routine the MCU performs the handshake with the

camera. If it succeeds then it'll start receiving the data and will store it in an array.

```
void interrupt_high_priority ISRH(void) {
    INTERRUPT_GLOBAL_H = 0; // Disattivo la chiamata interrupt
    wh_app = WREG; // Salvo i registri su variabili di appoggio
    sh_app = STATUS;
    //RC5STAbits.SPEN = 0; //disabilito UART arduino

    if (PIR3bits.RC2IF == 1 && RC2REG == 'c') {
//        leggi_omvcam();
        TX2REG = 'c';
        int datiRx = 0;
        unsigned int contatoreattesa = 0;
        while(datiRx < 7 && contatoreattesa < TEMPO_ATTESA_OMVCAM) {
            while(PIR3bits.RC2IF == 0 && contatoreattesa < TEMPO_ATTESA_OMVCAM) {
                contatoreattesa++;
            };

            if(contatoreattesa < TEMPO_ATTESA_OMVCAM ){
                tempCamera[datiRx] = RC2REG;
                contatoreattesa = 0;
                datiRx++;
                cicliErroriCam = 0;
            }
        }

        STATUS = sh_app;
        WREG = wh_app;
        INTERRUPT_GLOBAL_H = 1;
    }
}
```

High priority interrupt routine, received data from camera

In case the ball or the goals aren't found all their respective values will be saved as "-1", used as a "not found" flag in the performance conditions.

The second highest priority communication is the one with the secondary **MCU**. It calls a low priority interrupt which pauses all operations of the main program, except for the camera communication. Here the line sensor's status is copied by the main controller. Once that is done the interrupt routine ends but a flag in the performance process is raised.

```

int sens_attivi = 0;
int val_tot = 0;
for (int i = 0; i < 5; i++) {
    if (sens_linea[i] == 1) {
        switch (i) {
            case 0:
                val_tot += 15;
                break;
            case 1:
                val_tot += 90;
                break;
            case 2:
                val_tot += 180;
                break;
            case 3:
                val_tot += 270;
                break;
            case 4:
                val_tot += 345;
                break;
        }
        sens_attivi++;
    }
}

int val_media = 0;
switch (sens_attivi) {
    case 1:
        val_media = val_tot;
        break;
    case 2:
        val_media = val_tot * 0.50;
        break;
    case 3:
        val_media = val_tot * 0.33;
        break;
    case 4:
        val_media = val_tot * 0.25;
        break;
    case 5:
        val_media = val_tot * 0.2;
        break;
}
val_media += 180;
if (val_media > 360) {
    val_media = val_media % 360;
}
goto FINETEST;

FINETEST: ;
unsigned int cm = 0;
INTERRUPT_LIN_FLAG = 0; // Azzero flag interrupt
INTERRUPT_GLOBAL_L = 1; // Attivo interrupt basso LINEA
while (cm < TEMPO_MANOVRA_INTERRUPT) {
    leggi_bussola_calcola_err(); // Leggo la bussola e
    pid(); // Serve per aumentare l'errore della bussola
    direzioni[direzione(val_media)] ();
    cm++;
    __delay_ms(2);
}

```

This flag activates the “reentry” condition which calculates the right maneuver for avoiding the out of bounds.

This algorithm uses the known sensor's position to calculate at which angle the robot crossed the line. If multiple sensors crossed the line it will sum their angle position and then calculate the average based on the number of active sensors.

Lastly it will rotate the resulting angle of 180° in order to obtain and perform the opposite maneuver to stay inbounds.

5.2.2 Ball following algorithm

Originally, the robot used an array of pre-calculated directions. Obviously, this didn't encompass all 360 degrees but only a subset; consequently, the **ball-following algorithm** lacked fluidity. Since the algorithm calculates the trajectory to intercept the ball by multiplying the ball's direction by a variable K , we developed the `go()` function, which accepts degrees as an input.

This approach was initially discarded because trigonometric calculations are extremely resource-intensive for an **8-bit microcontroller**. We then pivoted to **Look-Up Tables (LUT)**, which consumed nearly all of our main MCU's memory. Now, when the robot boots up, it calculates an array of 91 sine values. By leveraging trigonometric properties, these values are used to calculate V_X and V_Y , which are then applied to the motor configuration.

```
void go(int direzione) {
    pwm1 = 0;
    pwm2 = 0;
    pwm3 = 0;
    pwm4 = 0;

    if (direzione < 0) direzione = 360 + direzione;
    else if (direzione >= 360) direzione = direzione % 360;
    lastDir = direzione;

    int direzione_cos;
    if(direzione >= 0 && direzione <= 90) {
        direzione_cos = (90-direzione);
        vx = (sins[direzione_cos]);
        vy = (sins[direzione]);
    } else if(direzione > 90 && direzione <= 180) {
        direzione = 90 - (direzione - 90);
        direzione_cos = (90-direzione);
        vx = (-sins[direzione_cos]);
        vy = (sins[direzione]);
    } else if(direzione > 180 && direzione <= 270) {
        direzione = (direzione - 180);
        direzione_cos = (90-direzione);
        vx = (-sins[direzione_cos]);
        vy = (-sins[direzione]);
    } else if(direzione > 270 && direzione <= 360) {
        direzione = 90 - (direzione - 270);
        direzione_cos = (90-direzione);
        vx = (sins[direzione_cos]);
        vy = (-sins[direzione]);
    }
}
```

go() fx (only VX and VY

calculation)

5.3. Secondary MCU

The secondary **MCU** is tasked with reading the 10 analog line sensors. Its code is written entirely in assembly language. This controller also drives an OLED **Display** which is used for debug and tuning purposes. The processor has a tuning algorithm which generates the thresholds of activation for the lines. This is achieved by summing the measured value on the field and on the line, then averaging it. This threshold is saved inside the micro EEPROM. This is useful since it saves us from the hustle of tuning the lines each time we power the robot.

This controller isn't just limited to "calling" the main one once a line is crossed. Inside the assembly code there are some conditions and decisions that help the reentry of the robot. First of all the process calls interrupt when 2 or more external sensors are activated or when just a single internal one has been triggered. This helps the robot recover from nasty spots such as the corners, where it could get confused and exit.

Second, the assembly code tries to figure out the direction of the line crossing. This flag is communicated to the main **MCU** but it hasn't been implemented in the line management algorithm.

```
;Primo gruppo AN0 AN1 AV_SX Ind EXT
TARA_N
;canale 0 nero

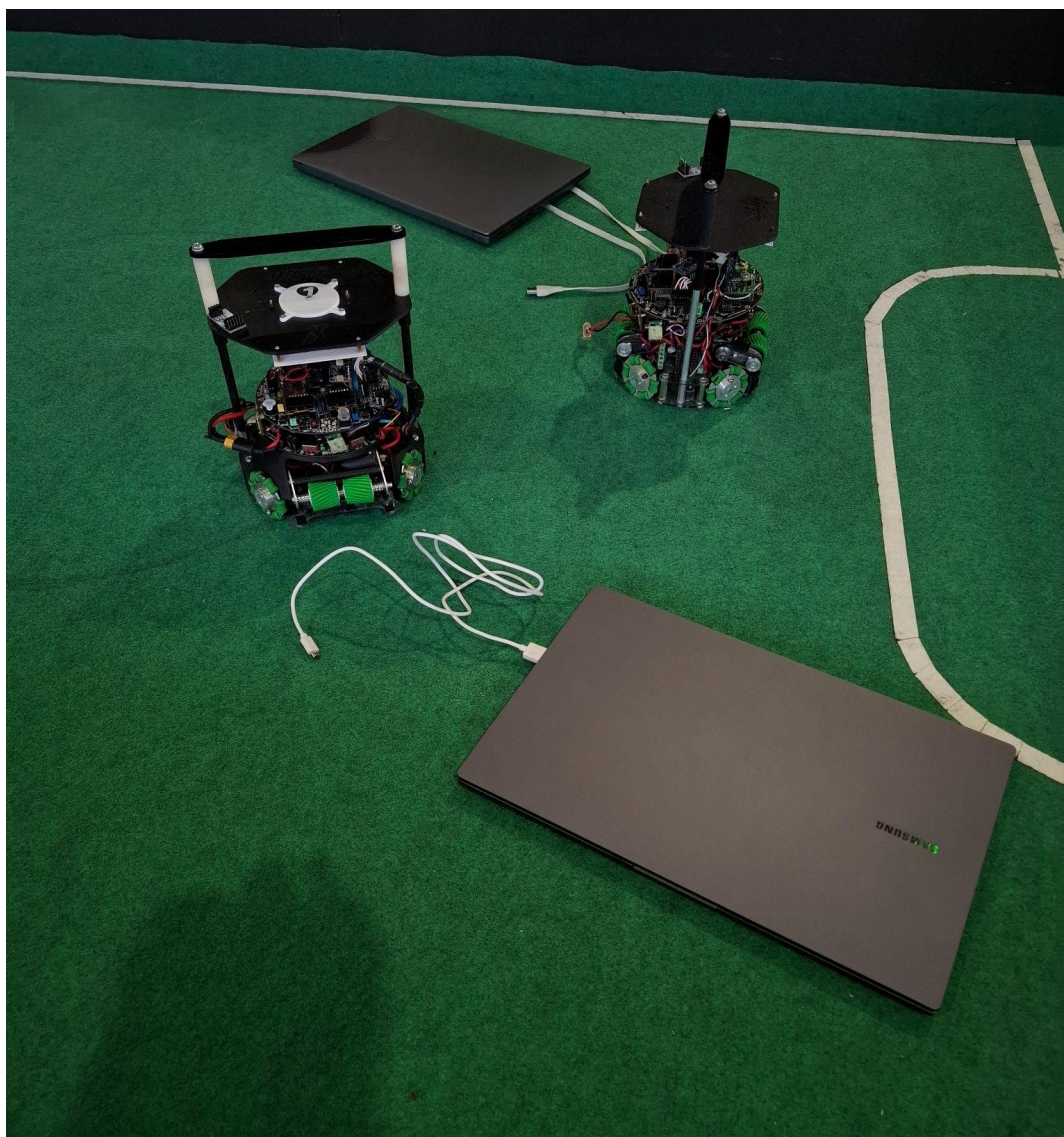
CALL    SEL_CH0      ;selezione canale analogico 0
CALL    ACQUIS        ;acquisizione valore
MOVWF   MIN1          ;memorizzazione nella variabile
MOVWF   TEMP_EXT      ;la subroutine cifre utilizza la variabile temp
CALL    CIFRE
movlw   XS1
movwf   x
movlw   YS1
movwf   y
call    ScriviSoglia
;canale 1 nero

CALL    SEL_CH1
CALL    ACQUIS
MOVWF   MIN2
MOVWF   TEMP_EXT
CALL    CIFRE
movlw   XS2
movwf   x
movlw   YS2
movwf   y
call    ScriviSoglia

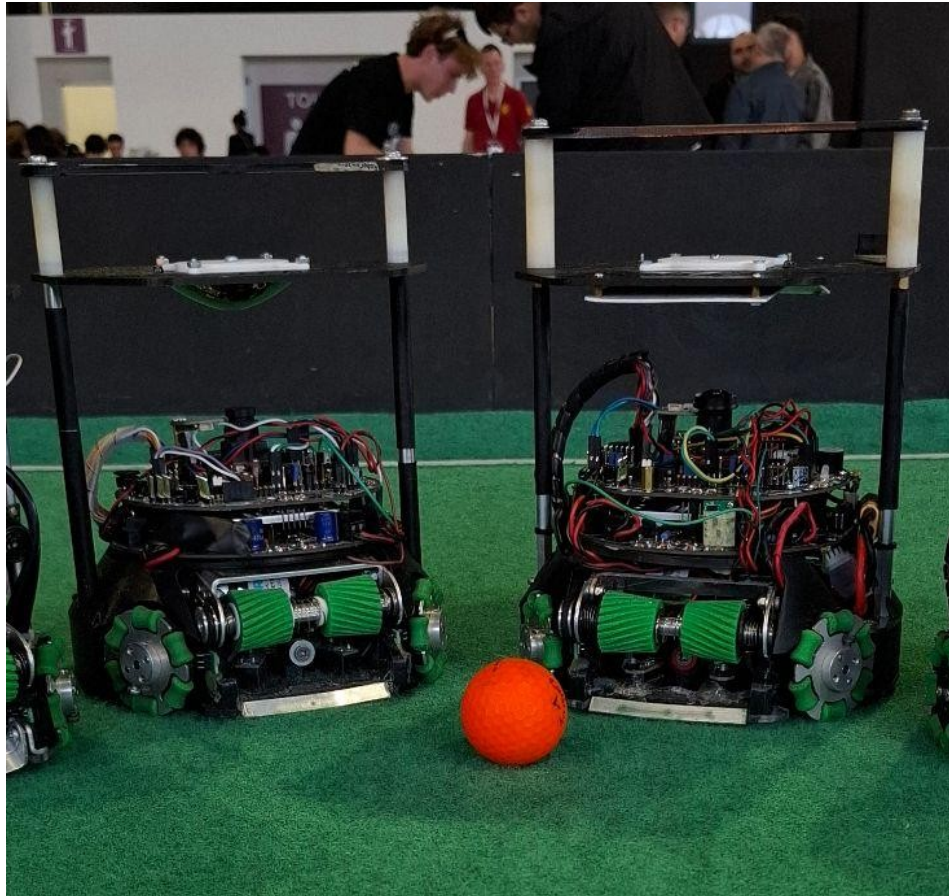
BTFSC   TARA_LIN      ;aspetto che venga premuto PORTB,0 per tarare gli altri due _INT
GOTO    TARA_N
MOVLW   100
CALL    MDELAY
```

Tuning routine written in assembly language

6. GALLERY



Robot Viper X and Viper IX



Viper IX and Viper X on the field at the Italian tournament